Week 5 at a glance

We will be learning and practicing to:

- Clearly and unambiguously communicate computational ideas using appropriate formalism. Translate across levels of abstraction.
  - Translating between symbolic and English versions of statements using precise mathematical language
  - Using appropriate signpost words to improve readability of proofs, including 'arbitrary' and 'assume'

Mon

- Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.

Mon
  - Writing the converse, contrapositive, and inverse of a given conditional statement

Mon
  - Determining what evidence is required to establish that a quantified statement is true or false

Mon
  - Evaluating quantified statements about finite and infinite domains

- Apply proof strategies, including direct proofs and proofs by contradiction, and determine whether a proposed argument is valid or not.

Wed/Fri
  - Identifying the proof strategies used in a given proof

Mon
  - Identifying which proof strategies are applicable to prove a given compound proposition based on its logical structure

Wed/Fri
  - Carrying out a given proof strategy to prove a given statement

Wed/Fri
  - Carrying out a universal generalization argument to prove that a universal statement is true

Wed/Fri
  - Using proofs as knowledge discovery tools to decide whether a statement is true or false

TODO:

Test 1 in the CBTF this week at your scheduled time! You will be required to show a physical photo identification in order to test. Electronic IDs and photos of IDs are not accepted. Additionally, you will only be allowed to bring your ID and a writing utensil into the testing room. You will have to store away all of your other belongings before entering the room. Please see the Triton Testing Center's website for more details.

Review quiz based on Week 4 class material (due Monday)

Midquarter feedback: please let us know what's working well for you and what isn't. `https://canvas.ucsd.edu/courses/77744/quizzes`

CC BY-NC-SA 2.0 Version October 23, 2026 (1)

Week 5 Monday: Nested Quantifiers and Proof Strategies

Recall the definitions: The set of RNA strands S is defined (recursively) by:

Basis Step:	$\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$
Recursive Step:	If $s \in S$ and $b \in B$, then $sb \in S$

where sb is string concatenation.

The function *rnalen* that computes the length of RNA strands in S is defined recursively by:

		$rnalen : S$	$\to \mathbb{Z}^+$
Basis Step:	If $b \in B$ then	$rnalen(b)$	$= 1$
Recursive Step:	If $s \in S$ and $b \in B$, then	$rnalen(sb)$	$= 1 + rnalen(s)$

The function *basecount* that computes the number of a given base b appearing in a RNA strand s is defined recursively by:

		$basecount : S \times B$	$\to \mathbb{N}$	
Basis Step:	If $b_1 \in B, b_2 \in B$	$basecount(\ (b_1, b_2)\ )$	$= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$	
Recursive Step:	If $s \in S, b_1 \in B, b_2 \in B$	$basecount(\ (sb_1, b_2)\ )$	$= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$	

Alternating nested quantifiers

Example 1:

$$\forall s \in S\ \exists n \in \mathbb{N}\ (\ basecount(\ (s, \mathtt{U})\ ) = n\ )$$

In English: For each strand, there is a nonnnegative integer that counts the number of occurrences of U in that strand.

Example 2:

$$\exists n \in \mathbb{N}\ \forall s \in S\ (\ basecount(\ (s, \mathtt{U})\ ) = n\ )$$

(name of variable; type of variable)

In English: There is a nonnnegative integer that counts the number of occurrences of U in every strand.

Are these statements true or false?

For each strand, can we find a witness number that works for that strand? Yes! So first example is true sentence.

Want single witness that works for all strands: impossible! So second example is false sentence.

CC BY-NC-SA 2.0 Version October 23, 2026 (2)

$BC((s,b,n))$ is T exactly when $basecount((s,b)) = n$.

$$\forall s \in S\ \exists b \in B\ (\ basecount(\ (s,b)\ ) = 3\ )$$

think about all strands to establish true

In English: For each RNA strand there is a base that occurs 3 times in this strand.

Write the negation and use De Morgan's law to find a logically equivalent version where the negation is applied only to the BC predicate (not next to a quantifier).

$$\neg \forall s \in S\ \exists b \in B (basecount((s,b)) = 3)$$
$$\equiv \exists s \in S\ \neg \exists b \in B (basecount((s,b)) = 3)$$
$$\equiv \exists s \in S\ \forall b \in B (basecount((s,b)) \neq 3)$$

need to find one witness

Is the original statement **True** or **False**?

Counterexample to original statement? Try AA

an RNA strand so in domain, need to check $\exists b \in B(basecount((AA,b)) = 3)$

$b \in B$	$basecount((AA,b)) = 3$? (ordered pair input, function application)	$basecount((AA,b)) \neq 3$?
A	$2 = 3$? F	T
C	$0 = 3$? F	T
U	$0 = 3$? F	T
G	$0 = 3$? F	T

so $\exists b \in B(basecount((AA,b)) = 3)$ is false (because all domain element are not witnesses)

and $\forall b \in B(basecount((AA,b)) \neq 3)$ is true

Suppose $P(x)$ is a predicate over a domain D.

1. To translate the statement "There are at least two elements in D where the predicate P evaluates to true", we could write

$$\exists x_1 \in D\, \exists x_2 \in D\, (P(x_1) \wedge P(x_2) \wedge \neg(x_1 = x_2))$$

$\exists x_1 \in D \exists x_2 \in D (P(x_1) \wedge P(x_2)) \equiv \exists x_1 \in D (P(x_1))$

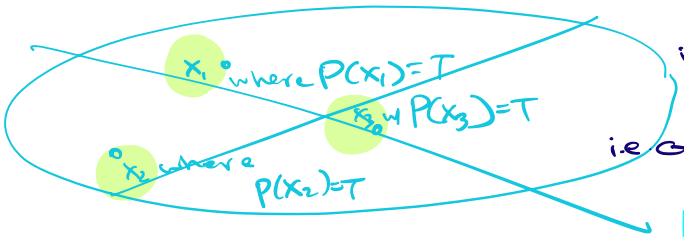
2. To translate the statement "There are at most two elements in D where the predicate P evaluates to true", we could write

universal conditional

$$\forall x_1 \in D\, \forall x_2 \in D\, \forall x_3 \in D\ (\ (\ P(x_1) \wedge P(x_2) \wedge P(x_3)\ ) \rightarrow (\ x_1 = x_2 \vee x_2 = x_3 \vee x_1 = x_3\ )\ )$$

Why? universal property constrains domains

labels need



Can't have hyp true and conc false

i.e. can't have $P(x_1) \wedge P(x_2) \wedge P(x_3) \wedge \neg(_ \vee _ \vee _)$

i.e. can't have $P(x_1) \wedge P(x_2) \wedge P(x_3) \wedge x_1 \neq x_2 \wedge x_2 \neq x_3 \wedge x_1 \neq x_3$

CC BY-NC-SA 2.0 Version October 23, 2026 (3)

Proof strategies

When a predicate $P(x)$ is over a **finite** domain:

- To show that $\forall x P(x)$ is true: check that $P(x)$ evaluates to T at each domain element by evaluating over and over. This is called "Proof of universal by **exhaustion**".
- To show that $\forall x P(x)$ is false: find a **counterexample**, a domain element where $P(x)$ evaluates to F.
- To show that $\exists x P(x)$ is true: find a **witness**, a domain element where $P(x)$ evaluates to T.
- To show that $\exists x P(x)$ is false: check that $P(x)$ evaluates to F at each domain element by evaluating over and over. DeMorgan's Law gives that $\neg\exists x P(x) \quad \equiv \quad \forall x \neg P(x)$ so this amounts to a proof of universal by exhaustion.

New! Proof by universal generalization: To prove that $\forall x\, P(x)$ is true, we can take an arbitrary element e from the domain of quantification and show that $P(e)$ is true, without making any assumptions about e other than that it comes from the domain.
An **arbitrary** element of a set or domain is a fixed but unknown element from that set.

New! Proof of conditional by direct proof: To prove that the conditional statement $p \to q$ is true, we can assume p is true and use that assumption to show q is true.

New! Proof of conditional by contrapositive proof: To prove that the implication $p \to q$ is true, we can assume q is false and use that assumption to show p is also false.

New! Proof of disjuction using equivalent conditional: To prove that the disjunction $p \vee q$ is true, we can rewrite it equivalently as $\neg p \to q$ and then use direct proof or contrapositive proof.

New! Proof by Cases: To prove q, we can work by cases by first describing all possible cases we might be in and then showing that each one guarantees q. Formally, if we know that $p_1 \vee p_2$ is true, and we can show that $(p_1 \to q)$ is true and we can show that $(p_2 \to q)$, then we can conclude q is true.

New! Proof of conjunctions with subgoals: To show that $p \wedge q$ is true, we have two subgoals: subgoal (1) prove p is true; and, subgoal (2) prove q is true.

To show that $p \wedge q$ is false, it's enough to prove that $\neg p$.
To show that $p \wedge q$ is false, it's enough to prove that $\neg q$.

CC BY-NC-SA 2.0

Week 5 Wednesday: Proof Strategies and Sets

Definitions:

A **set** is an unordered collection of elements. When A and B are sets, $A = B$ (set equality) means

$$\forall x(x \in A \leftrightarrow x \in B)$$

When A and B are sets, $A \subseteq B$ ("A is a **subset** of B") means

$$\forall x(x \in A \rightarrow x \in B)$$

When A and B are sets, $A \subsetneq B$ ("A is a **proper subset** of B") means

$$(A \subseteq B) \wedge (A \neq B)$$

To prove that one set is a subset of another, e.g. to show $A \subseteq B$:

To prove that two sets are equal, e.g. to show $A = B$:

CC BY-NC-SA 2.0

Example: $\{43, 7, 9\} = \{7, 43, 9, 7\}$

Prove or **disprove**: $\{\texttt{A}, \texttt{C}, \texttt{U}, \texttt{G}\} \subseteq \{\texttt{AA}, \texttt{AC}, \texttt{AU}, \texttt{AG}\}$

Prove or **disprove**: For some set B, $\emptyset \in B$.

Prove or **disprove**: For every set B, $\emptyset \in B$.

Prove or **disprove**: The empty set is a subset of every set.

CC BY-NC-SA 2.0

Prove or **disprove**: The empty set is a proper subset of every set.

Prove or **disprove**: $\{4, 6\} \subseteq \{n \mid \exists c \in \mathbb{Z}(n = 4c)\}$

Prove or **disprove**: $\{4, 6\} \subseteq \{n \textbf{ mod } 10 \mid \exists c \in \mathbb{Z}(n = 4c)\}$

Consider ..., an **arbitrary** .... **Assume** ..., we **want to show** that .... Which is what was needed, so the proof is complete □.

or, in other words:

Let ... be an **arbitrary** .... **Assume** ..., **WTS** that ... **QED**.

CC BY-NC-SA 2.0

Cartesian product: When A and B are sets,

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

Example: $\{43, 9\} \times \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} \times \emptyset =$

Union: When A and B are sets,

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

Example: $\{43, 9\} \cup \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} \cup \emptyset =$

Intersection: When A and B are sets,

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

Example: $\{43, 9\} \cap \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} \cap \emptyset =$

Set difference: When A and B are sets,

$$A - B = \{x \mid x \in A \wedge x \notin B\}$$

Example: $\{43, 9\} - \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} - \emptyset =$

Disjoint sets: sets A and B are disjoint means $A \cap B = \emptyset$

Example: $\{43, 9\}, \{9, \mathbb{Z}\}$ are not disjoint

Example: The sets $\mathbb{Z}$ and $\emptyset$ are disjoint

Power set: When U is a set, $\mathcal{P}(U) = \{X \mid X \subseteq U\}$

Example: $\mathcal{P}(\{43, 9\}) =$

Example: $\mathcal{P}(\emptyset) =$

CC BY-NC-SA 2.0

Let $W = \mathcal{P}(\{1, 2, 3, 4, 5\})$

Example elements in W are:

Prove or **disprove**: $\forall A \in W \, \forall B \in W \; (A \subseteq B \; \rightarrow \; \mathcal{P}(A) \subseteq \mathcal{P}(B))$

Prove or **disprove**: $\forall A \in W \, \forall B \in W \; (\mathcal{P}(A) = \mathcal{P}(B) \; \rightarrow \; A = B)$

Prove or **disprove**: $\forall A \in W \, \forall B \in W \, \forall C \in W \; (A \cup B = A \cup C \; \rightarrow \; B = C)$

CC BY-NC-SA 2.0

Week 5 Friday: Proof Strategies and Numbers

Facts about numbers

We now have propositional and predicate logic that can help us express statements about any domain. We will develop proof strategies to craft valid argument for proving that such statements are true or disproving them (by showing they are false). We will practice these strategies with statements about sets and numbers, both because they are familiar and because they can be used to build cryptographic systems. Then we will apply proof strategies more broadly to prove statements about data structures and machine learning applications.

1. Addition and multiplication of real numbers are each commutative and associative.

2. The product of two positive numbers is positive, of two negative numbers is positive, and of a positive and a negative number is negative.

3. The sum of two integers, the product of two integers, and the difference between two integers are each integers.

4. For every integer x there is no integer strictly between x and $x + 1$,

5. When x, y are positive integers, $xy \geq x$ and $xy \geq y$.

Factoring

Definition: When a and b are integers and a is nonzero, a **divides** b means there is an integer c such that $b = ac$.

Symbolically, $F(\ (a, b)\ ) =$ and is a predicate over the domain _____

Other (synonymous) ways to say that $F(\ (a, b)\ )$ is true:

a is a **factor** of b a is a **divisor** of b b is a **multiple** of a $a|b$

When a is a positive integer and b is any integer, $a|b$ exactly when b **mod** $a = 0$

When a is a positive integer and b is any integer, $a|b$ exactly when $b = a \cdot (b$ **div** $a)$

CC BY-NC-SA 2.0

Translate these quantified statements by matching to English statement on right.

$\exists a \in \mathbb{Z}^{\neq 0}$ ($F($ (a, a) $)$)	Every nonzero integer is a factor of itself.
$\exists a \in \mathbb{Z}^{\neq 0}$ ($\neg F($ (a, a) $)$)	No nonzero integer is a factor of itself.
$\forall a \in \mathbb{Z}^{\neq 0}$ ($F($ (a, a) $)$)	At least one nonzero integer is a factor of itself.
$\forall a \in \mathbb{Z}^{\neq 0}$ ($\neg F($ (a, a) $)$)	Some nonzero integer is not a factor of itself.

Claim: Every nonzero integer is a factor of itself.

Proof:

Prove or **Disprove**: There is a nonzero integer that does not divide its square.

Prove or **Disprove**: Every positive factor of a positive integer is less than or equal to it.

Claim: Every nonzero integer is a factor of itself and every nonzero integer divides its square.

CC BY-NC-SA 2.0

Definition: an integer n is **even** means that there is an integer a such that $n = 2a$; an integer n is **odd** means that there is an integer a such that $n = 2a + 1$. Equivalently, an integer n is **even** means $n \mod 2 = 0$; an integer n is **odd** means $n \mod 2 = 1$. Also, an integer is even if and only if it is not odd.

Definition: An integer p greater than 1 is called **prime** means the only positive factors of p are 1 and p. A positive integer that is greater than 1 and is not prime is called composite.

Extra examples: Use the definition to prove that 1 is not prime, 2 is prime, 3 is prime, 4 is not prime, 5 is prime, 6 is not prime, and 7 is prime.

True or False: The statement "There are three consecutive positive integers that are prime."

Hint: These numbers would be of the form $p, p + 1, p + 2$ (where p is a positive integer).

Proof: We need to show _____

True or False: The statement "There are three consecutive odd positive integers that are prime."

Hint: These numbers would be of the form $p, p + 2, p + 4$ (where p is an odd positive integer).

Proof: We need to show _____

CC BY-NC-SA 2.0